# Differentiate Code or Not

What an agent reliability team can deliver with only an API endpoint, versus with access to the code — and how to add tracing to a client's agent with the least friction.
October 2026

> ### The short version
>
> **Endpoint only** tells a client *what* their agent gets wrong.
> **Code access** (tracing inside the agent) tells them *why* and *where*, and lets them watch real users.
>
> You do not need code access to win the first deal. Start with an endpoint audit, then ask for a small, reviewable change.

### Contents


1. Where Phoenix can run
2. Endpoint vs code access: what the client gets
3. The middle ground (gray box)
4. What has to change in the agent
5. How to get the change into a client's codebase
6. A ready-to-use instruction file for an AI coding assistant
7. Privacy and safety controls
8. Do we need code access? Business view
9. Pitch: service tiers

## 1. Where Phoenix can run

Localhost is only for development. Phoenix always sits behind the agent's **backend**, never inside the website or mobile app.

| Option | Where it runs | Best for | Data leaves the client? |
|---|---|---|---|
| Local | `phoenix serve` on a laptop | Development, demos | No |
| Self-hosted | Client's own server or cloud (Docker, Kubernetes) | Clients strict about data | No |
| Phoenix Cloud / Arize AX | Hosted by Arize | Fastest setup | Yes, to Arize |

Moving between them is a settings change only: the collector address (`PHOENIX_COLLECTOR_ENDPOINT`) and, for cloud, an API key.

## 2. Endpoint vs code access: what the client gets

| Deliverable | Endpoint only (black box) | Code access (traces from inside) |
|---|---|---|
| Is the final answer correct? | Yes | Yes |
| Is the answer made up (hallucination)? | Partly — vs expected answers only | Yes — vs the real tool data it saw |
| Total response time | Yes | Yes, split: model vs each tool |
| Error rate | Only HTTP errors | Every failed step, even hidden ones |
| Which tool it chose | No | Yes |
| Arguments passed to tools | No | Yes |
| What tools returned | No | Yes |
| Loops / repeated calls | No | Yes |
| Tokens and cost per chat | No | Yes |
| The exact prompt the model saw | No | Yes |
| Root cause of a failure | Educated guess | Shown step by step |
| Monitoring real production users | No | Yes |
| Safety / red-team testing | Yes | Yes, plus where the guard failed |
| Load and speed testing | Yes | Yes, plus the slow step |
| Before/after comparison of a change | Answers only | Answers and behaviour |
| Client effort | None — an API key | ~10 lines of code, 1–2 settings |

> **Typical endpoint-only findings:** wrong answers, made-up facts, unsafe replies, wrong language, slow replies, inconsistent answers to the same question.
> **Extra findings with code access:** "it picked the wrong tool", "it passed yesterday's date", "the search tool returned nothing and the model invented a clinic", "73% of time is spent in the second model call".

## 3. The middle ground (gray box)

When a client will not open the code, these options recover most of the value.

| Method | What we can see | Client effort |
|---|---|---|
| **Their engineer adds our tracing snippet**, data goes to their own Phoenix | Everything | One small change they review |
| **LLM proxy** (LiteLLM, Helicone, Portkey): change only the model base URL | Prompts, tool calls the model requests, tool results sent back, tokens, model latency | One setting, no code |
| **Debug metadata** in API responses on staging (e.g. tools used) | Tool names, step count | Small |
| **Export of existing logs** (Datadog, LangSmith, CloudWatch) | Whatever they already log | None |
| **Staging copy** of the agent we can instrument | Everything, no production risk | Medium |

## 4. What has to change in the agent

Adding tracing is small and does not change what the agent does. The work is the same shape for any framework:

1. **Install** the tracing packages.
2. **Register** tracing once at startup: project name and Phoenix address.
3. **Turn on** the framework's instrumentation, often automatic.
4. **Set** 1–3 environment variables.
5. **Verify**: send one chat, see one trace in Phoenix.

### Per framework (what "turn on" means)

| Agent built with | Change needed | Size |
|---|---|---|
| Vercel AI SDK (JS) — e.g. Vetmed | `@arizeai/phoenix-otel` + `@ai-sdk/otel`, call `register()` and `registerTelemetry()` once | 1 file, ~30 lines |
| OpenAI SDK (Python or JS) | OpenInference OpenAI instrumentor, auto-captures every call | ~5 lines |
| LangChain / LangGraph | OpenInference LangChain instrumentor | ~5 lines |
| LlamaIndex, CrewAI, Anthropic, Bedrock, etc. | Matching OpenInference instrumentor | ~5 lines |
| Python, any of the above | `phoenix.otel.register(auto_instrument=True)` picks up installed instrumentors | 2 lines |
| Custom / homemade agent loop | Wrap the model call and each tool call in a span manually | 20–60 lines |

### Reference: what we changed in the Vetmed agent

| File | Change |
|---|---|
| `services/chatbot/phoenix-tracing.js` (new) | Registers Phoenix and AI SDK telemetry once; fails safe if Phoenix is down. |
| `services/chatbot/complete-stream-text.js` | Calls the setup once and adds `telemetry: { isEnabled: true }` to the model call. |
| `package.json` | Added the two tracing packages. |
| Agent logic, prompts, tools, frontend | **Untouched.** |

```
// phoenix-tracing.js (core of it)
const { register } = require('@arizeai/phoenix-otel');
const { OpenTelemetry } = await import('@ai-sdk/otel');
const provider = register({ projectName: 'vetmed-chatbot', url: ENDPOINT });
registerTelemetry(new OpenTelemetry({
  tracer: provider.getTracer('vetmed-chatbot'),
  headers: false,            // never record API keys
}));
```

### Zero-code or near-zero-code options

- **Startup flag:** load the tracing file before the app, e.g. `node --import ./instrumentation.js server.js`. App code stays untouched; only the start command changes.
- **Python auto-instrumentation:** one `register(auto_instrument=True)` at the entry point.
- **LLM proxy:** change the model base URL in config. No code, but less detail on tool internals.

## 5. How to get the change into a client's codebase

| Method | How it works | Client trust needed | Speed |
|---|---|---|---|
| **A. Instruction doc** | We send a 1–2 page guide; their engineer follows it. | Low | 1–2 hours |
| **B. AI coding prompt file** | We send a file (section 6). Their engineer opens Cursor, Claude Code or Copilot, points it at the file, and the assistant makes the change. Engineer reviews the diff. | Low | 15–30 min |
| **C. Patch / pull request from us** | We get read-only repo access (or a single file) and send a PR they review and merge. | Medium | Fast |
| **D. Pairing session** | A 1-hour call; their engineer drives, we guide. | Low | 1 hour |
| **E. Full repo access** | We do it end to end. | High | Fastest |

**Recommended:** B with D as backup. The client never gives us their code, their engineer stays in control, and the change takes minutes.

**Always:** the change is approved by the client's engineering or security lead and covered in the contract. Never get a change merged by going around the people who own the system — it destroys trust and can breach the contract.

## 6. Ready-to-use instruction file for an AI coding assistant

Ship this as `ADD_AGENT_TRACING.md`. Their engineer tells their assistant: "*Follow ADD_AGENT_TRACING.md.*"

```
# Add Phoenix tracing to this agent

## Goal
Record every agent run (user input, each model call, each tool call
with arguments and result, final answer, latency, errors) to Phoenix.
Do NOT change agent behaviour, prompts, tools, or responses.

## Steps
1. Find the agent framework (Vercel AI SDK, OpenAI SDK, LangChain,
   LlamaIndex, CrewAI, or custom loop) and the backend entry point.
2. Install the matching Phoenix / OpenInference tracing packages.
   Do not upgrade any existing package versions.
3. Create ONE new file (e.g. tracing.js / tracing.py) that:
   - registers Phoenix with project name from PHOENIX_PROJECT_NAME
     and address from PHOENIX_COLLECTOR_ENDPOINT
   - enables the framework's instrumentation
   - never records request headers or API keys
   - catches all errors so the app still runs if Phoenix is down
4. Load that file once at startup (entry point or start command).
5. Add to .env.example (not .env):
   PHOENIX_COLLECTOR_ENDPOINT=
   PHOENIX_PROJECT_NAME=
   PHOENIX_API_KEY=        # only for hosted Phoenix
6. Keep the diff minimal. List every file you changed.

## Verify
- Start Phoenix, start the app, send one chat.
- One trace appears with LLM and TOOL steps.
- App behaves exactly as before with Phoenix stopped.

## Do not
- Edit prompts, tool logic, models, or business code.
- Commit secrets.
- Record user messages or outputs if the client asked to hide them
  (use the framework's hide/record-inputs option instead).
```

## 7. Privacy and safety controls

| Concern | Control |
| --- | --- |
| Customer chats leaving the company | Self-host Phoenix in the client's cloud |
| Personal data in traces | Hide inputs/outputs (OpenInference hide settings, or AI SDK `recordInputs: false`), or redact before export |
| API keys leaking | Header capture off |
| Production risk | Start on staging; tracing is fail-safe; one setting turns it off |
| Who can see data | Phoenix login and roles; we get read-only accounts |
| Legal | NDA + data processing agreement listing exactly what is collected |

## 8. Do we need code access? Business view

**No — not to start.** Asking for code on day one is a common deal-killer. Companies are usually less worried about code than about **data** (customer chats) and **control** (who changes production).

- **Lead with value that needs no access:** an endpoint audit finds real problems in about two weeks.
- **Ask for a small change, not access:** "Your engineer runs this file; data stays in your Phoenix."
- **Remove the data fear:** self-hosted Phoenix, hidden inputs, a DPA.
- **Fallback:** the LLM proxy needs only a config change.
- **Upgrade naturally:** the audit report itself shows what they are missing ("root cause unknown without tracing").

## 9. Pitch: service tiers

| Tier | Access needed | What we deliver | Duration |
|---|---|---|---|
| **1. Audit** | API endpoint + test account | 100–300 test chats; quality, safety, speed, consistency report | 1–2 weeks |
| **2. Diagnose** | Tracing on staging (method B) | Root causes, tool accuracy, cost breakdown, prioritised fixes | 2–3 weeks |
| **3. Monitor** | Tracing in production | Monthly report, regression tests on every release, alerts | Ongoing |

> **One-line pitch:** "We start with nothing but your API. If you want to know *why* it fails, your engineer runs one file — your data never leaves your systems."